FT9xx Series 32-Bit Programming Guide

Software Engineering Team – paul.jiao@brtchip.comf – Version V0.1, 2026-02-11

Table of Contents

1. 概述 (Introduction)
2. 寄存器映射 (Register Maps)
3. 外设编程实例 (Peripheral Programming)
    3.1. UART 初始化
4. 内容复用 (Modular Content)
    4.1. CTRL Register (32-bit)
        4.1.1. Bitfield Diagram
        4.1.2. Field Description
        4.1.3. C Example (Read-Modify-Write)
5. 公式与计算
6. 寄存器描述 32-bit Register Example
    6.1. GPIO_MODE Register (0x00)
        6.1.1. Register Layout (32-bit)
        6.1.2. Field Definitions
        6.1.3. C Access Example
7. 波形设计 (Timing Diagrams)
8. 架构框图 architect diagrams

1. 概述 (Introduction)

本文档旨在为嵌入式软件工程师提供系列 MCU 的底层驱动开发指南。

> 在操作高压外设前，请确保 VDD 处于 3.3V 稳定状态。

2. 寄存器映射 (Register Maps)

Register Name	Offset	Access	Description
GPIO_MODE	0x00	R/W	配置 GPIO 引脚模式
GPIO_ODATA	0x04	R/W	输出数据寄存器
GPIO_IDATA	0x08	R	输入数据寄存器

3. 外设编程实例 (Peripheral Programming)

3.1. UART 初始化

以下是基于 **C** 语言 的初始化代码示例：

```c
#include "mcu_uart.h"

void uart_init(uint32_t baudrate) {
    UART->BAUD = SystemCoreClock / baudrate; // 设置波特率
    UART->CTRL |= UART_ENABLE;               // 使能串口
}
```

4. 内容复用 (Modular Content)

4.1. CTRL Register (32-bit)

Base Address: `0x4002_0000`
Offset: `0x00`
Access: R/W
Reset: `0x0000_0000`

4.1.1. Bitfield Diagram

Table 1. Bits 31..16

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED (31:24)								SPEED (23:16)							

Table 2. Bits 15..0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PULL (15:12)				MODE (11:8)				EN (7)	INTEN (6)	KEY (5:0)					

4.1.2. Field Description

Bits	Field	Reset	Description
31:24	RESERVED	0x00	Reserved. Keep at reset value.
23:16	SPEED	0x00	Output speed. `0=Low`, `1=Med`, `2=High`, `3=VeryHigh`
15:12	PULL	0x0	Pull config. `0=None`, `1=Up`, `2=Down`

Bits	Field	Reset	Description
11:8	MODE	0x0	Mode. `0=Input`, `1=Output`, `2=AF`, `3=Analog`
7	EN	0	Enable. `0=Disable`, `1=Enable`
6	INTEN	0	Interrupt enable. `0=Off`, `1=On`
5:0	KEY	0x00	Write key. Must write `0x2A` to update MODE/PULL/SPEED.

4.1.3. C Example (Read-Modify-Write)

```c
```

```c
#include <stdint.h>

#define REG32(addr) (*(volatile uint32_t *)(addr))

#define CTRL_BASE   0x40020000UL
#define CTRL_OFF    0x00UL
#define CTRL        REG32(CTRL_BASE + CTRL_OFF)

#define CTRL_SPEED_Pos   16u
#define CTRL_SPEED_Msk   (0xFFu << CTRL_SPEED_Pos)

#define CTRL_PULL_Pos    12u
#define CTRL_PULL_Msk    (0xFu  << CTRL_PULL_Pos)

#define CTRL_MODE_Pos    8u
#define CTRL_MODE_Msk    (0xFu  << CTRL_MODE_Pos)

#define CTRL_EN_Pos      7u
#define CTRL_EN_Msk      (1u    << CTRL_EN_Pos)

#define CTRL_INTEN_Pos   6u
#define CTRL_INTEN_Msk   (1u    << CTRL_INTEN_Pos)

#define CTRL_KEY_Pos     0u
#define CTRL_KEY_Msk     (0x3Fu << CTRL_KEY_Pos)
#define CTRL_KEY_VALUE   0x2Au

static inline void ctrl_config(uint32_t mode, uint32_t pull, uint32_t speed, uint32_t en)
{
    uint32_t reg = CTRL;

    // Preserve reserved bits; only modify documented fields.
    reg &= ~(CTRL_MODE_Msk | CTRL_PULL_Msk | CTRL_SPEED_Msk | CTRL_EN_Msk | CTRL_KEY_Msk);
    reg |= ((mode  & 0xFu)  << CTRL_MODE_Pos);
    reg |= ((pull  & 0xFu)  << CTRL_PULL_Pos);
    reg |= ((speed & 0xFFu) << CTRL_SPEED_Pos);
    reg |= ((en ? 1u : 0u)  << CTRL_EN_Pos);
    reg |= (CTRL_KEY_VALUE  << CTRL_KEY_Pos);

    CTRL = reg;
}
```

5. 公式与计算

对于 PWM 频率的计算，公式如下：

$$f_{PWM} = \frac{f_{clk}}{(Prescaler + 1) \times (Period + 1)}$$

use ASCII math, 公式如下:

$$f_{PWM} = \frac{f_{clk}}{(Presca \leq r + 1) \cdot (Period + 1)}$$

6. 寄存器描述 32-bit Register Example

6.1. GPIO_MODE Register (0x00)

6.1.1. Register Layout (32-bit)

Bits	Field	Reset	Description
31:16	RESERVED	0x0000	Reserved, must keep default value.
15:8	SPEED	0x00	Output speed selection.
7:4	PULL	0x0	Pull-up / pull-down configuration.
3:0	MODE	0x0	Pin mode selection.

6.1.2. Field Definitions

- **MODE** (Bits [3:0])
  - `0x0` = Input
  - `0x1` = Output
  - `0x2` = Alternate Function
  - `0x3` = Analog
- **PULL** (Bits [7:4])
  - `0x0` = No pull
  - `0x1` = Pull-up
  - `0x2` = Pull-down
- **SPEED** (Bits [15:8])
  - `0x00` = Low
  - `0x01` = Medium
  - `0x02` = High
  - `0x03` = Very High

6.1.3. C Access Example

C

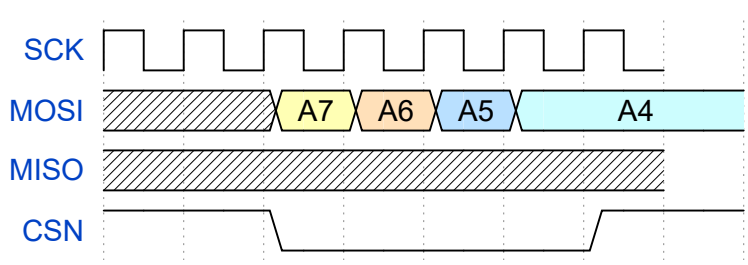
```c
#include <stdint.h>

#define GPIO_BASE      (0x40020000UL)
#define GPIO_MODE_OFF  (0x00UL)
#define GPIO_MODE      (*(volatile uint32_t *)(GPIO_BASE + GPIO_MODE_OFF))

// Bit positions / masks
#define GPIO_MODE_MODE_Pos   0
#define GPIO_MODE_MODE_Msk   (0xFUL << GPIO_MODE_MODE_Pos)

#define GPIO_MODE_PULL_Pos   4
#define GPIO_MODE_PULL_Msk   (0xFUL << GPIO_MODE_PULL_Pos)

#define GPIO_MODE_SPEED_Pos  8
#define GPIO_MODE_SPEED_Msk  (0xFFUL << GPIO_MODE_SPEED_Pos)

static inline void gpio_mode_set(uint32_t mode, uint32_t pull, uint32_t speed)
{
    uint32_t reg = GPIO_MODE;

    reg &= ~(GPIO_MODE_MODE_Msk | GPIO_MODE_PULL_Msk | GPIO_MODE_SPEED_Msk);
    reg |= ((mode  & 0xFUL)  << GPIO_MODE_MODE_Pos);
    reg |= ((pull  & 0xFUL)  << GPIO_MODE_PULL_Pos);
    reg |= ((speed & 0xFFUL) << GPIO_MODE_SPEED_Pos);

    GPIO_MODE = reg;
}

static inline uint32_t gpio_mode_get(void)
{
    return GPIO_MODE;
}
```

Notes

- This register is 32-bit aligned, so offsets increment by 4 bytes (0x00, 0x04, 0x08...).
- Always preserve **RESERVED** bits (read-modify-write) unless the spec says otherwise.

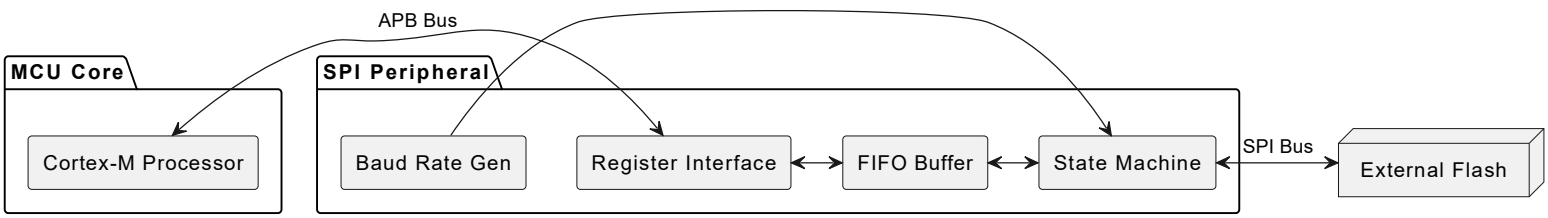
7. 波形设计 (Timing Diagrams)

以下是 SPI 数据传输的典型时序要求：



8. 架构框图 architect diagrams

This diagram is drawn by plantuml code.



It can be done by inserting the image :

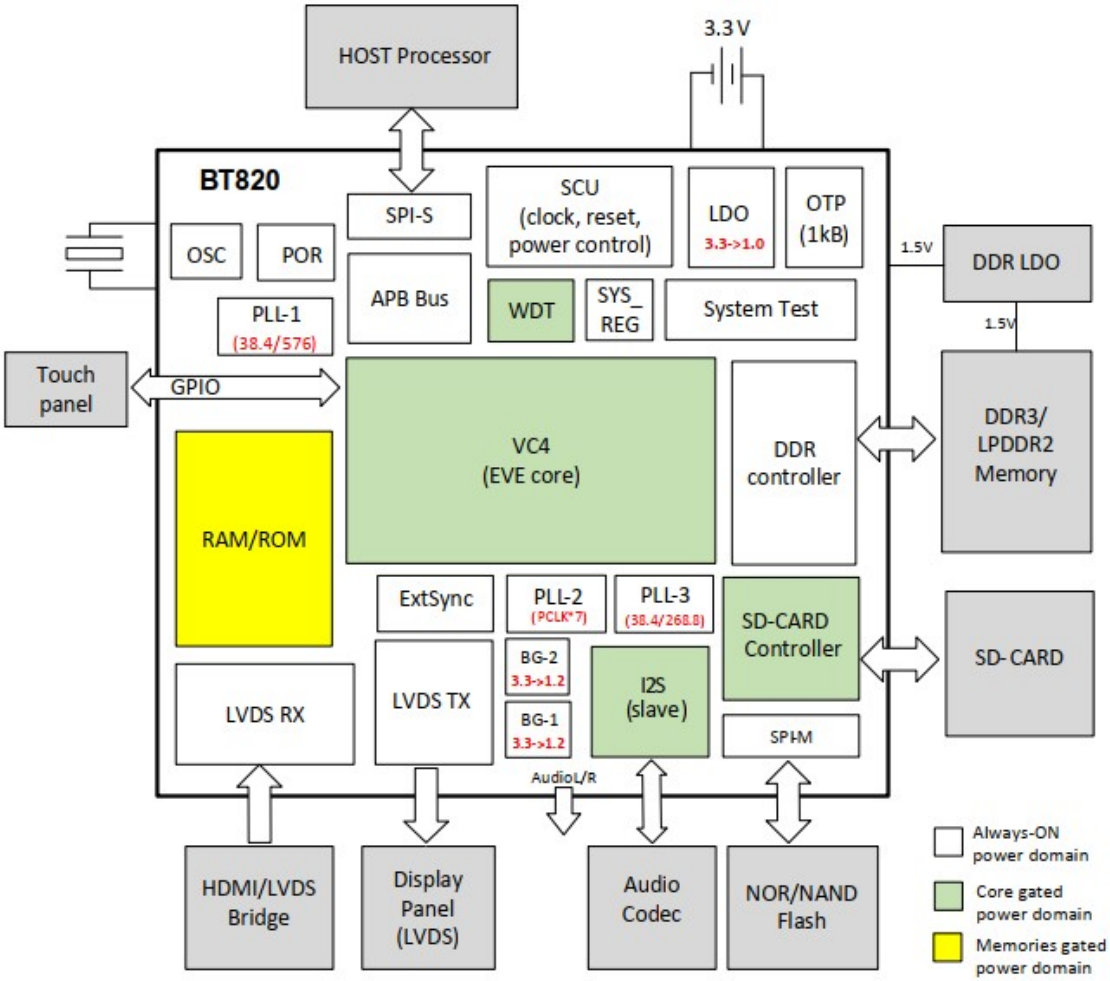


Figure 1. 系统架构框图

修订记录 (Revision History)

日期	版本	作者	变更说明
2026-02-12	v0.1	Paul Jiao	初稿：新增寄存器章节与位域图示
2026-02-15	v0.2	KH Low	修正章节编号与 TOC 显示

Version v0.1
Last updated 2026-02-12 03:30:16 UTC